Reviewer Pack — Minimal Order of Symplectic Leaves Correlation with Circuit …

Entry b0d258b56e52 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 02:12:05 UTC

Minimal Order of Symplectic Leaves Correlation with Circuit Monotone Complexity

Entry ID: `b0d258b56e52` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 02:12:05 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Boolean Circuits

Statement:

> For every d-regular graph G, the minimal order of symplectic leaves (m_order(G)) in its associated affine variety is linearly correlated with its circuit monotone complexity w_m(G), such that m_order(G) = Θ(w_m(G)).

Rationale (proposer's reasoning):

> Symplectic geometry provides a geometric framework to study algebraic varieties. By mapping graph structures into symplectic leaves, we may uncover hidden structural properties of circuits, potentially leading to new lower bounds on circuit complexity.

Taxonomy category: `SymplecticGeometryBooleanCircuits` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8272d691262e5763`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between m_order(G) and w_m(G) for all d-regular graphs G with n vertices in {5, …, 40} is greater than or equal to 0.8 AND the mean of the correlation coefficients across 30 seeds is less than or equal to 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"symplectic geometry" AND "Boolean circuits" AND "circuit monotone complexity"` - `"minimal order of symplectic leaves" AND d-regular graph AND "circuit monotone complexity"` - `"affine variety" IN Symplectic Geometry AND Boolean Circuit Complexity`

Top relevant hits considered: - [http://arxiv.org/abs/1612.04764v1] Cohomological aspects on complex and symplectic manifolds - [http://arxiv.org/abs/1002.3099v3] Tamed Symplectic forms and SKT metrics - [http://arxiv.org/abs/math/0601540v1] Symplectic forms and surfaces of negative square

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_d_regular(graph, d):
        degrees = [sum(1 for _ in neighbors) for _, neighbors in graph.items()]
        return all(degree == d for degree in degrees)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            return None
        graph = {i: set() for i in range(n)}
        edges_added = 0
        while edges_added < n * d // 2:
            u = random.randint(0, n - 1)
            v = random.choice(list(graph.keys()))
            if u != v and v not in graph[u]:
                graph[u].add(v)
                graph[v].add(u)
                edges_added += 1
        return graph

    def calculate_m_order(graph):
        # Placeholder for m_order calculation
        return len(graph)

    def calculate_w_m(graph):
        # Placeholder for w_m calculation
        return sum(len(neighbors) for _, neighbors in graph.items()) / len(graph)
```

```
    n = random.choice([5, 10, 15, 20, 30, 40])
    d = random.randint(2, min(n - 1, 3))
    graph = generate_d_regular_graph(n, d)

    if not is_d_regular(graph, d):
        return {
            "metric_name": "Pearson correlation coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    m_order = calculate_m_order(graph)
    w_m = calculate_w_m(graph)

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": (m_order, w_m),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("counterexample" not in r or r["counterexample"] == "" for r in results):
        mean_corr = sum(r["metric_value"][0] / r["instances_tested"] for r in results) / len(resul
        std_corr = (sum((r["metric_value"][0] / r["instances_tested"] - mean_corr) ** 2 for r in r
        support_fraction = sum(1 for r in results if "conjecture_holds" and r["conjecture_holds"])

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_corr:.2f} std={std_corr:.2f} support_fraction={su
        else:
            print(f"RESULT: FALSIFIED counterexample=\"not_enough_support\" first_failing_seed=1")
    else:
        first_failing_seed = next(result["seed"] for result in results if "counterexample" in resu
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74b6eb10.py", line 79, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74b6eb10.py", line 51, in run_trial
    if not is_d_regular(graph, d):
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74b6eb10.py", line 22, in is_d_regular
    degrees = [sum(1 for _ in neighbors) for _, neighbors in graph.items()]
                                                             ^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'items'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the Pearson correlation coefficient could not be calculated. As a result, it is inconclusive whether the conjecture holds. | next: Investigate and fix the crash in the test code to allow for the calculation of the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13196
2	preregistration	ollama_remote	glm4:latest	0	0	19999
3	novelty	ollama_remote	glm4:latest	0	0	13999
4	novelty	ollama_remote	glm4:latest	0	0	9413
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18881
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27481
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16373
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20827
9	judge	ollama_remote	glm4:latest	0	0	17025

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 157193 ms total latency. Provider mix: {‘ollama_remote’: 9}

(full prompt+response transcripts available in `research/audit/b0d258b56e52.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b0d258b56e52.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b0d258b56e52.tar.gz` (if generated)